The End of a Craft?

"Craftsmanship names an enduring, basic human impulse, the desire to do a job well for its own sake." — Richard Sennett, *The Craftsman* (2008)[1]

If the agent does the building, what's left that only a person can do?

Everyone is asking this out loud now and most of the answers are either panic or a sales pitch.

The honest answer is the work that can't be prompted: the judgment before the prompt exists and after the agent has run.[2] The agent is extraordinary at building what it's told to build. It has no opinion about whether that was the right thing to build, or whether the system it's being added to is becoming something coherent or something nobody can hold.

That judgment has a name once it's shared. Direction. And it's the part of the craft the agent doesn't touch.

Direction is easy to mistake for vision, which sounds like a slogan on a wall. It's more concrete than that. It's the running answer to "where does this need to go," held closely enough that the next decision can be checked against it. Whether today's choice still makes sense six months from now. Whether the system is becoming one thing or three.

The agent has none of it, and not because it isn't capable enough yet. It optimizes the request in front of it, which is what it's for. Asked whether this is a good implementation of what it was told, it will often answer better than a person could. It has nothing of its own to say, though, about whether the system should be heading there at all, because that question is about the whole history of the system and where it's meant to end up, not the task on the screen. That picture lives in people who have watched the system become what it is.

[1] Richard Sennett, *The Craftsman* (Yale University Press, 2008).

[2] This is "Discernment" in the 4D Framework for AI Fluency — Delegation, Description, Discernment, Diligence — developed by Rick Dakan and Joseph Feller with Anthropic (aifluencyframework.org). The framework names the individual competencies for working with AI; this book is about the shared understanding a team builds before those competencies have anything reliable to act on.

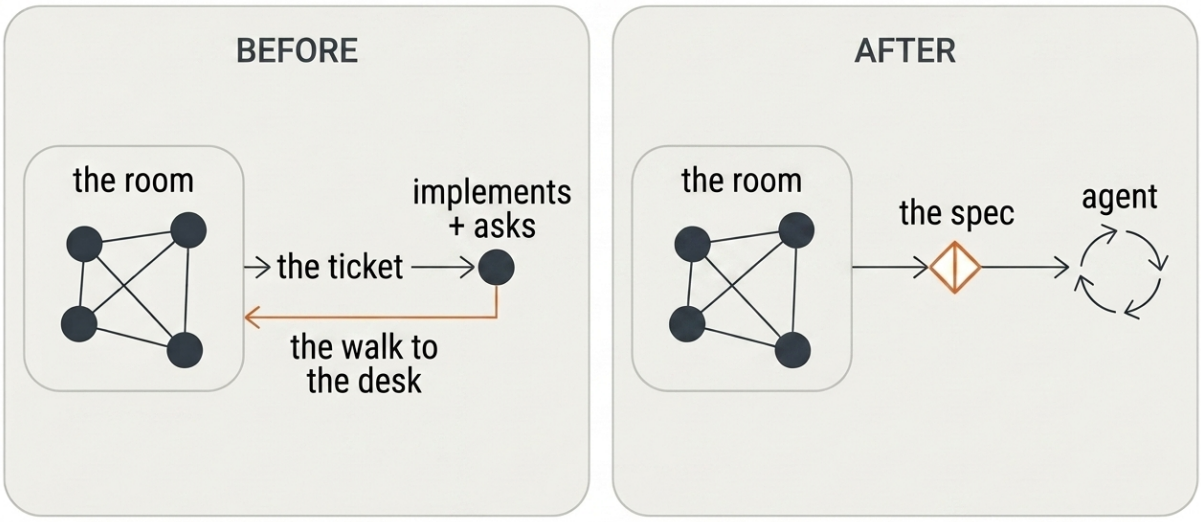

Figure 1: Before: the room hands off a ticket, and the implementer walks back to ask what it means. After: the room converges on a spec, and the agent runs from that directly.

It's fair to push back that the agent isn't blind to that history anymore. Memory persists across sessions, and context files like AGENTS.md[3] let a team write the picture down and hand it over. That's real, but it moves the work rather than removing it. Someone still has to write that picture, keep it true, and hold it in common. A memory kept in one person's local setup is the old privatization in new clothes: the drift between two copies is now written down, and still invisible to anyone but them. An agent can hold direction. It can't decide it: choosing what's worth building is a judgment about the world outside the context window. It can't notice when the direction goes stale, because staleness is measured against conversations the agent wasn't in. And it can't know whether the copy it holds is the one the team is actually steering by, because that fact lives in the team, not in any file. Three gaps, one shape: direction is a relationship to a team, not a string in a repository.

And more of the picture isn't automatically better. A context swollen with every past decision can drag the agent down the way a session left running too long does, because the bloat buries the signal, and only a person can tell which part is still load-bearing and which has turned to noise. A fresh, well-aimed start often beats a long one carrying everything it ever accumulated.

[3]AGENTS.md is an open format for agent instruction files (agents.md), adopted across a range of coding agents.

The judgment shows up before anything gets built. Which approach, and not the other. What scope is worth it. What "good enough" means for this team, this week. The agent builds what it's told. It doesn't decide whether the thing is worth building at all. Deciding that is direction work, and it's irreducibly human.

Over one Christmas I built a Rust evaluation engine that compiled to WebAssembly, a small program in a language I'd never written a line of. The same open-source specification had a separate implementation in five languages: Python, Java, .NET, TypeScript, and Rust, and each was quietly making slightly different decisions at the edges. One engine compiled to WebAssembly meant one set of decisions, every language inheriting the same behavior. The agent wrote every line. I couldn't have. What I brought was the judgment around it: that the disagreement between the five was the actual problem to solve, that a single shared engine was the way to end it, and that a rough holiday prototype was enough to prove the idea. The agent built. I decided what was worth building, why, and how we'd know it worked.

That division is the whole shape of the work now. The agent on implementation. The person on direction.

It was always the more interesting half. The craft was never the typing. It was the judgment the typing crowded out. Implementation was just where the time went. Take the time away, and what's left is the part that took years to learn.

Direction has two failure modes, and the agent will flag neither.

The first is that nobody holds it at all. The work still gets done: tickets close, features ship, the agent never tires. But every decision is made against a local picture, and the local pictures slowly stop agreeing. The system becomes one thing in one corner and a different thing in another. Months later someone reaches a choice that no longer fits and goes looking for the reason it was made, and there isn't one, because the person who made it was answering a smaller question at the time. Nothing announces this. It's the most expensive kind of drift exactly because no single step ever looked wrong.

The second is subtler, because it looks like the problem is solved: one person holds the direction, and a direction held by one person is just an opinion. The moment someone builds against a different picture of where the system is going, the work diverges quietly, because nobody is wrong, they're just pointed differently. A Northstar only does its job when the team is steering by the same one. And a shared direction doesn't come from one person thinking clearly. It comes from a room.

That's the argument the rest of this book has to earn. For now it's enough to say the craft didn't end. One craft did, the implementation most engineers built their identity on. What replaced it is the judgment that implementation used to crowd out.

So the craft survives, moved up to direction, the high, slow plane, where the system is headed over months. But the agent erodes judgment closer to the ground too: in what the tool quietly stops an engineer from noticing, one prompt at a time.